

Week 9: Backtesting, Risk Analysis & Taker Strategies

Assembling the full system – Milestone 3 (Coincall guest session)

North Carolina State University | Summer 2026

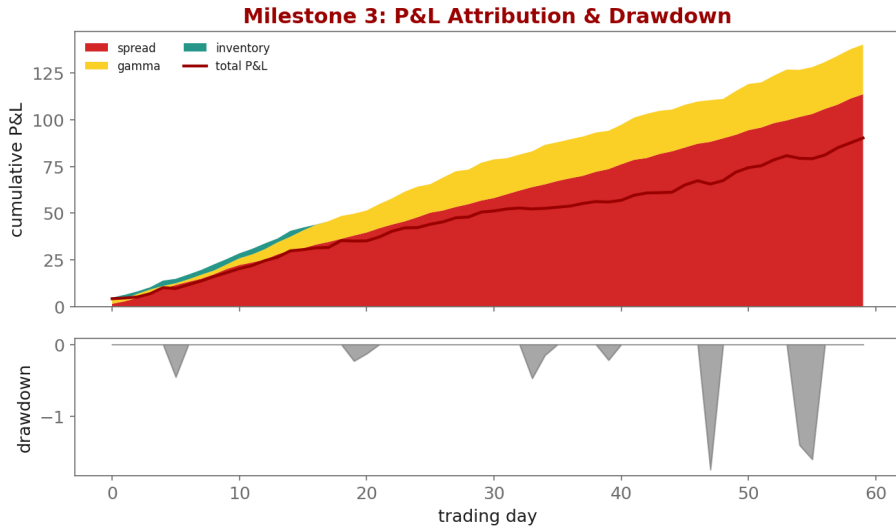
NC STATE UNIVERSITY

Industry Mentor: Dendi Suhubdy (Bitwyre) | Guest: Fenni Kang (Coincall)

- ▶ Build an event-driven options backtest
- ▶ Produce a full P&L attribution and risk metrics
- ▶ Understand taker strategies and order-flow toxicity

- ▶ Event-driven simulation with deterministic replay
- ▶ Fill simulator; state of inventory and Greeks
- ▶ Pitfalls: look-ahead, unrealistic fills, expiry handling
- ▶ Sharpe with block-bootstrap confidence intervals

- ▶ When and why takers cross the spread
- ▶ Signals behind aggressive flow: momentum, liquidations, news
- ▶ What taker behaviour means for the maker's toxicity



$$\widehat{SR} = \frac{\bar{r}}{s_r} \sqrt{252}, \quad \text{CI via block bootstrap on } \{r_t\}$$

Why: Daily Sharpe is annualized by $\sqrt{252}$, but a point estimate over a few months is noisy and serially correlated. We resample BLOCKS of returns (preserving autocorrelation) to get an honest confidence interval – a high Sharpe with a wide CI is not a real edge.

- ▶ Full options L2 + trades + perp + funding for 3 months:
 - ▶ `coincall_btc_options_l2_2025Q4.parquet / ..._trades...`
 - ▶ `coincall_eth_options_l2_2025Q4.parquet / ..._trades...`
 - ▶ `coincall_btc_eth_perp_l2_2025Q4.parquet ; coincall_funding_rates...`
- ▶ Klines for regime context: GET
`/open/option/market/kline/history/v1/{name}?period=h1`
- ▶ Trade tape carries mark IV at trade -> calibrate the fill simulator

Cross-check exact paths at <https://docs.coincall.com/>

```

import numpy as np
def attribute(spread, inventory, gamma, hedging):
    total = spread + inventory + gamma + hedging
    assert abs(total.sum() - (spread.sum()+inventory.sum()+
        gamma.sum()+hedging.sum())) < 1e-6 # must sum to total
    return dict(spread=spread, inventory=inventory, gamma=gamma,
        hedging=hedging, total=total)

def bootstrap_sharpe(r, block=5, n=2000, seed=0):
    rng = np.random.default_rng(seed); T = len(r); out = []
    for _ in range(n):
        idx = (rng.integers(0, T-block, T//block)[: ,None] + np.arange(block)).ravel()
        s = r[idx]; out.append(s.mean()/s.std()*np.sqrt(252))
    return np.percentile(out, [2.5, 50, 97.5]) # CI for the Sharpe

```

- ▶ Cartea et al., Ch. 10-11; Bailey-Lopez de Prado (2014); Lopez de Prado (2018)